

RL Framework for Agent Execution Optimization

KavAI Home Assignment — Optional Section

Fatemeh Doudi

May 10, 2026

Contents

1	Overview	2
2	Formulation A: Bandit Formulation	2
3	Formulation B: Stop/Continue MDP (Primary)	2
3.1	State	2
3.2	Action	2
3.3	Reward Function	3
3.4	Policy	3
4	Training Setup	3
4.1	Data	3
4.2	Algorithm	3
4.3	Training Loop	3
4.4	Hyperparameters	3
5	Evaluation	4
5.1	Baselines.	4

1 Overview

The goal of this framework is to train a policy that learns the optimal execution control for the sensor query agent. specifically, whether to continue iterating or stop at each step. Rather than fixing `max_iterations` as a static hyperparameter, we treat it as a decision variable learned through reinforcement learning, balancing correctness against cost.

We present two formulations: a simpler bandit formulation (Formulation A) and a full MDP formulation (Formulation B).

2 Formulation A: Bandit Formulation

In this simpler formulation, the agent observes the query and selects $n \in \{1, 2, 3, 4, 5\}$ before execution begins. This is a contextual bandit problem where one decision is made per query with no state transitions, as the action does not influence subsequent states. A lightweight action network $f_\theta(\mathbf{e}_q) \rightarrow n$ maps the query embedding directly to the number of iterations.

- **State:** query embedding $\mathbf{e}_q \in \mathbb{R}^d$
- **Action:** $n \in \{1, 2, 3, 4, 5\}$, selected by a learned network f_θ
- **Reward:** $R = \mathbf{1}[\text{correct}] - \lambda \cdot n$

This formulation commits to n before seeing any intermediate results, making it unable to adapt to tool errors or partial outputs observed mid-execution.

3 Formulation B: Stop/Continue MDP (Primary)

3.1 State

At each iteration t , the state captures both the query context and the current execution status:

$$s_t = [\mathbf{e}_q, t, \mathbf{1}_{\text{error}}, n_{\text{records}}]$$

where:

- $\mathbf{e}_q \in \mathbb{R}^d$: query embedding
- $t \in \mathbb{N}$: current iteration count
- $\mathbf{1}_{\text{error}} \in \{0, 1\}$: whether the last tool call returned an error
- $n_{\text{records}} \in \mathbb{N}$: number of records returned by the last filter call

3.2 Action

At each step the policy selects a binary action:

$$a_t \in \{\text{stop}, \text{continue}\}$$

If $a_t = \text{stop}$, the agent returns the current answer. If $a_t = \text{continue}$, the agent runs another iteration. The action is selected by a lightweight policy network $\pi_\theta(a_t | s_t)$. Execution also terminates if a hard cap T_{max} is reached.

3.3 Reward Function

The reward is received at termination and balances correctness with efficiency:

$$R = \underbrace{\mathbf{1}[\text{correct}]}_{\text{correctness}} - \lambda \cdot \underbrace{t}_{\text{cost}}$$

where $\mathbf{1}[\text{correct}] \in \{-1, +1\}$ is defined as:

- +1: agent’s filtered records match ground truth
- -1: wrong records returned or empty result when records exist

and λ is a tradeoff hyperparameter controlling the accuracy-efficiency balance. This simple setting doesn’t have intermediate credit assignment.

3.4 Policy

The policy is a learned function:

$$\pi_{\theta}(a_t | s_t) = \text{MLP}_{\theta}(s_t)$$

A shallow MLP takes the state vector s_t as input and outputs a probability distribution over $\{\text{stop}, \text{continue}\}$. The policy can be trained end-to-end using any policy gradient based algorithms.

4 Training Setup

4.1 Data

A labeled dataset of queries with ground truth answers is required. We assume a dataset of N query-answer pairs, split 80/20 into train and evaluation sets. Ground truth consists of the expected filtered records for each valid query, and a “decline” label for invalid queries.

4.2 Algorithm

Since reward is only received at termination with no intermediate credit assignment, standard policy gradient methods are directly applicable. The policy network is a 2-layer MLP activations trained to maximize expected return.

4.3 Training Loop

1. Sample a batch of queries from the training set
2. For each query, run the agent under the current policy π_{θ} , collecting the trajectory $(s_0, a_0, s_1, a_1, \dots, s_T, R)$
3. Update θ using the PPO clipped objective
4. Repeat until convergence

4.4 Hyperparameters

Key hyperparameters include the tradeoff coefficient $\lambda \in \{0.05, 0.1, 0.2\}$, learning rate α , and hard iteration cap $T_{\max}$.

5 Evaluation

The policy is evaluated on the held-out eval set using three metrics:

- **Accuracy**: percentage of queries where the returned records match ground truth.
- **Accuracy-efficiency tradeoff curve** : by sweeping λ , we can show the Pareto frontier of the learned policy.

5.1 Baselines.

The learned policy is compared against two baselines: fixed `max_iterations` = 1 and fixed `max_iterations` = 5. A well-trained policy should achieve accuracy close to the $n = 5$ baseline while using significantly fewer iterations on average.